Name: Dwijesh Mistry
Class / Batch: TY4 – B
Roll No: 38

K MEANS CLUSTERING

Program Code:

```
import random
import re
import os
import time
from typing import List, Optional, Tuple

def parse_int_list(text: str) -> List[int]:
    parts = [p for p in re.split(r"[\s,]+", text.strip()) if p]
    if not parts:
        raise ValueError("No numbers provided.")
    return [int(p) for p in parts]

def mean(values: List[int]) -> float:
    return sum(values) / len(values)

def assign_points_to_centroids(data: List[int], centroids: List[float]) -> List[int]:
    assignments: List[int] = []
    for x in data:
        best_idx = 0
        best_dist = abs(x - centroids[0])
        for i in range(1, len(centroids)):
            dist = abs(x - centroids[i])
            if dist < best_dist:
                best_dist = dist
                best_idx = i
        assignments.append(best_idx)
    return assignments

def build_clusters(data: List[int], assignments: List[int], k: int) -> List[List[int]]:
    clusters: List[List[int]] = [[] for _ in range(k)]
    for x, idx in zip(data, assignments):
```

```
            clusters[idx].append(x)
    return clusters

def reinit_empty_clusters(
    clusters: List[List[int]],
    centroids: List[float],
    rng: random.Random,
    data: List[int],
) -> Tuple[List[List[int]], List[float], bool]:
    changed = False
    for i, cluster in enumerate(clusters):
        if cluster:
            continue
        # If a cluster becomes empty, re-initialize its centroid to a random data
point.
        centroids[i] = float(rng.choice(data))
        changed = True
    return clusters, centroids, changed

def kmeans_1d(
    data: List[int],
    k: int,
    *,
    seed: Optional[int] = None,
    max_iter: int = 100,
    verbose: bool = True,
) -> Tuple[List[List[int]], List[float]]:
    if k <= 0:
        raise ValueError("K must be a positive integer.")
    if k > len(data):
        raise ValueError("K cannot be greater than the number of input values.")
    if max_iter <= 0:
        raise ValueError("max_iter must be positive.")

    rng = random.Random(seed)
    # Pick K random starting centroids from the input numbers (integers).
    start_indices = rng.sample(range(len(data)), k)
    centroids = [float(data[i]) for i in start_indices]

    if verbose:
```

```
        print("Input data:", data)
        print(f"K = {k}")
        if seed is not None:
            print(f"Seed = {seed}")
        print("Initial centroids:", [int(c) if c.is_integer() else c for c in centroids])

    prev_assignments: Optional[List[int]] = None

    for it in range(1, max_iter + 1):
        assignments = assign_points_to_centroids(data, centroids)
        clusters = build_clusters(data, assignments, k)
        clusters, centroids, reinit_happened = reinit_empty_clusters(clusters,
centroids, rng, data)
        if reinit_happened:
            assignments = assign_points_to_centroids(data, centroids)
            clusters = build_clusters(data, assignments, k)

        new_centroids: List[float] = []
        for i, cluster in enumerate(clusters):
            if cluster:
                new_centroids.append(mean(cluster))
            else:
                new_centroids.append(centroids[i])

        if verbose:
            print(f"\nIteration {it}:")
            for i, cluster in enumerate(clusters):
                cluster_sorted = sorted(cluster)
                centroid_str = f"{new_centroids[i]:.4f}".rstrip("0").rstrip(".")
                print(f"  Cluster {i + 1}: {cluster_sorted} | mean = {centroid_str}")

        if prev_assignments is not None and assignments == prev_assignments:
            centroids = new_centroids
            if verbose:
                print("\nConverged: cluster sets unchanged.")
            return clusters, centroids

        prev_assignments = assignments
        centroids = new_centroids
```

```
    if verbose:
        print("\nStopped: reached max_iter without full convergence.")
    return clusters, centroids

def main() -> None:
    run_id = time.strftime("%Y-%m-%d %H:%M:%S")
    print(f"Run started ({run_id}) | PID={os.getpid()}")
    raw_numbers = input("Enter integers (space/comma-separated): ").strip()
    data = parse_int_list(raw_numbers)

    raw_k = input("Enter K (number of clusters): ").strip()
    if not raw_k:
        raise ValueError("K is required.")
    k = int(raw_k)

    raw_seed = input("Optional seed (press Enter to skip): ").strip()
    seed = int(raw_seed) if raw_seed else None

    clusters, centroids = kmeans_1d(data, k, seed=seed, max_iter=100,
verbose=True)

    print("\nFinal result:")
    for i, cluster in enumerate(clusters):
        cluster_sorted = sorted(cluster)
        centroid_str = f"{centroids[i]:.4f}".rstrip("0").rstrip(".")
        print(f"  Cluster {i + 1}: {cluster_sorted} | mean = {centroid_str}")

if __name__ == "__main__":
    main()
```

Output:

```
Enter integers (space/comma-separated): 1 2 3 5 6 7 10 11 12 13 14 15 20 22 23 24
Enter K (number of clusters): 3
Optional seed (press Enter to skip):
Input data: [1, 2, 3, 5, 6, 7, 10, 11, 12, 13, 14, 15, 20, 22, 23, 24]
K = 3
Initial centroids: [11, 12, 15]

Iteration 1:
  Cluster 1: [1, 2, 3, 5, 6, 7, 10, 11] | mean = 5.625
  Cluster 2: [12, 13] | mean = 12.5
  Cluster 3: [14, 15, 20, 22, 23, 24] | mean = 19.6667

Iteration 2:
  Cluster 1: [1, 2, 3, 5, 6, 7] | mean = 4
  Cluster 2: [10, 11, 12, 13, 14, 15] | mean = 12.5
  Cluster 3: [20, 22, 23, 24] | mean = 22.25

Iteration 3:
  Cluster 1: [1, 2, 3, 5, 6, 7] | mean = 4
  Cluster 2: [10, 11, 12, 13, 14, 15] | mean = 12.5
  Cluster 3: [20, 22, 23, 24] | mean = 22.25

Converged: cluster sets unchanged.

Final result:
  Cluster 1: [1, 2, 3, 5, 6, 7] | mean = 4
  Cluster 2: [10, 11, 12, 13, 14, 15] | mean = 12.5
  Cluster 3: [20, 22, 23, 24] | mean = 22.25
```